

Zadanie 3

1. Przygotowanie projektu za pomocą Spring Initializr - <https://start.spring.io/>
 - a) Ustalenie właściwości projektu, wersji SpringBoot 2.7.x, języka Java 17, narzędzia Maven
 - b) Ustalenie typu projektu - .war
 - c) Ustalenie nazw i pakietów aplikacji
 - d) Dodanie zależności: Web, Data JPA, H2
 - e) Wygenerowanie plików projektu

Project: ☐ Gradle - Groovy ☐ Gradle - Kotlin ☒ Java ☐ Kotlin ☐ Groovy

Language: ☒ Java ☐ Kotlin ☐ Groovy

Spring Boot: ☐ 3.3.0 (SNAPSHOT) ☐ 3.3.0 (M1) ☐ 3.2.3 (SNAPSHOT) ☒ 3.2.2 ☐ 3.1.9 (SNAPSHOT) ☐ 3.1.8

Project Metadata

Group:

Artifact:

Name:

Description:

Package name:

Packaging: ☐ Jar ☒ War

Java: ☐ 21 ☒ 17

Dependencies:

Spring Web **WEB**
Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

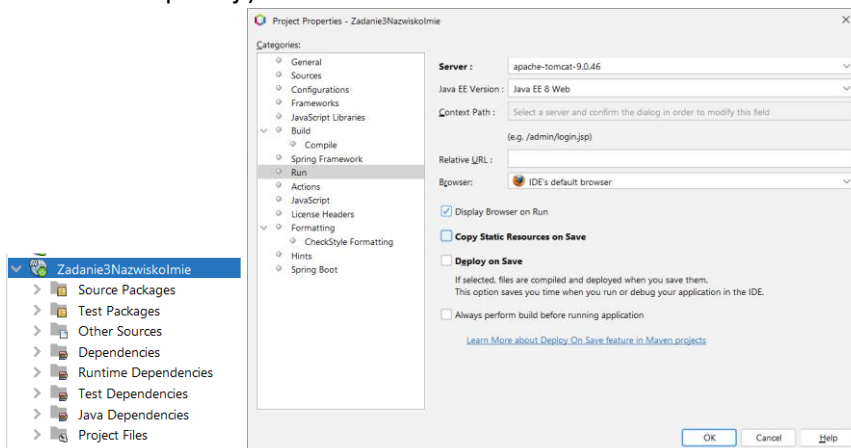
Spring Data JPA **JPA**
Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.

H2 Database **JPA**
Provides a fast in-memory database that supports JDBC API and R2DBC access, with a small (2mb) footprint. Supports embedded and server modes as well as a browser based console application.

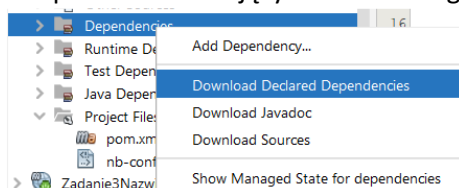
```
<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.7.9</version>
<relativePath/> <!-- lookup parent from repository -->
</parent>
<groupId>com.tss</groupId>
<artifactId>Zadanie3NazwiskoImie</artifactId>
<version>0.0.1-SNAPSHOT</version>
<packaging>war</packaging>
<name>Zadanie3NazwiskoImie</name>
<description>Demo project for Spring Boot</description>
<properties>
```

2. Wczytanie projektu do środowiska NetBeans

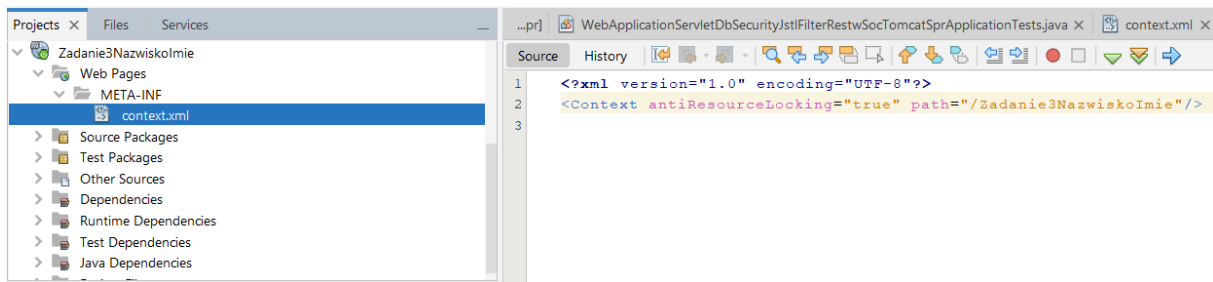
- a) Wykonanie budowanie projektu z opcją Build With Dependencies
- b) Sprawdzenie struktury projektu
- c) **Zmiana wersji SpringBoot na 2.7.9 w pliku pom.xml**
- d) Ustalenie serwera – apache-tomcat-9.0.46 (używanego w kursie)
- e) Odznaczenie opcji „Copy Static Resources...” i „Deploy on Save” (gdy problemy z redeploy aplikacji).



Uzupełnienie brakujących zależności gdy konieczne

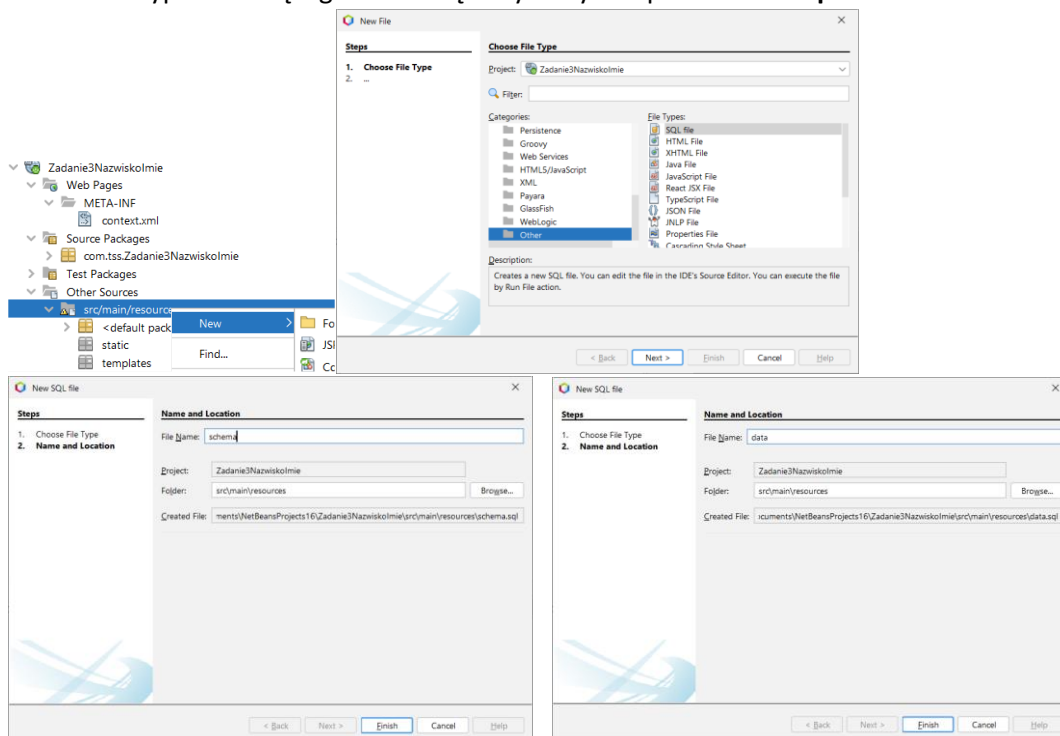


3. Ustawienie kontekstu aplikacji w pliku context.xml



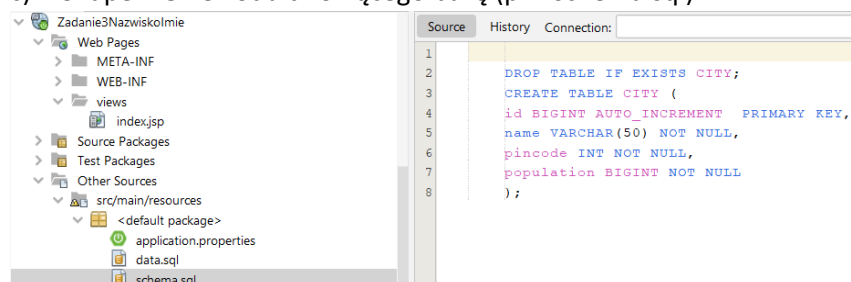
4. Dodanie i konfiguracja bazy danych H2 działającej w pamięci

a) Dodanie skryptu tworzącego strukturę bazy danych – plik **schema.sql**

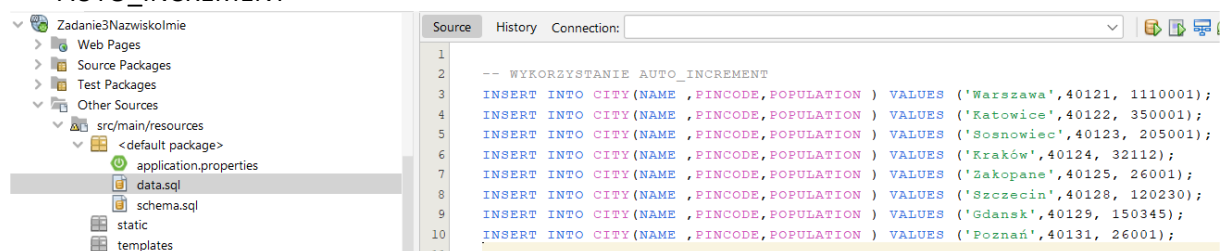


b) Dodanie skryptu inicjalizującego bazę danych – plik **data.sql**

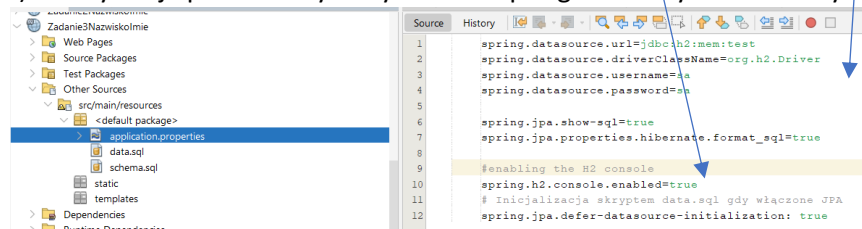
c) Uzupełnienie kodu tworzącego bazę (plik **schema.sql**)



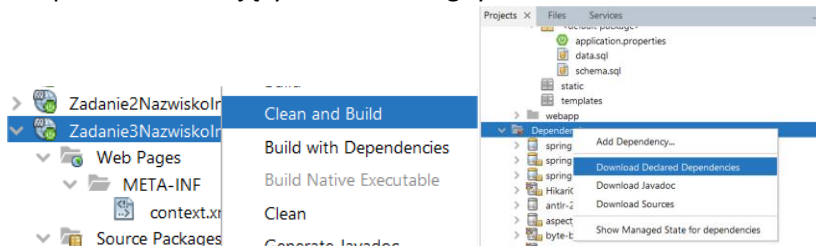
d) Uzupełnienie kodu inicjalizującego dane w bazie (plik **data.sql**) z wykorzystaniem opcji **AUTO_INCREMENT**



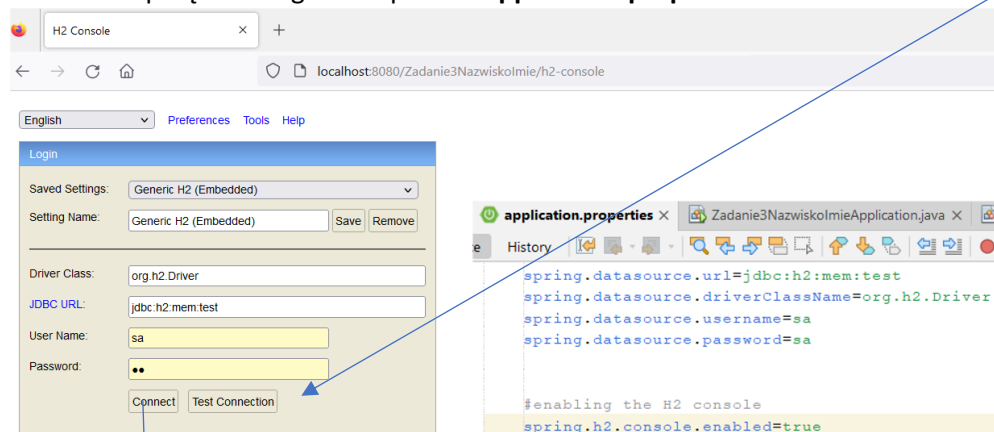
- e) Dodanie parametrów Spring Data w pliku application.properties
 f) Aktywacja panelu bazy danych H2 w Spring Boot i wymuszenie wykonania skryptów



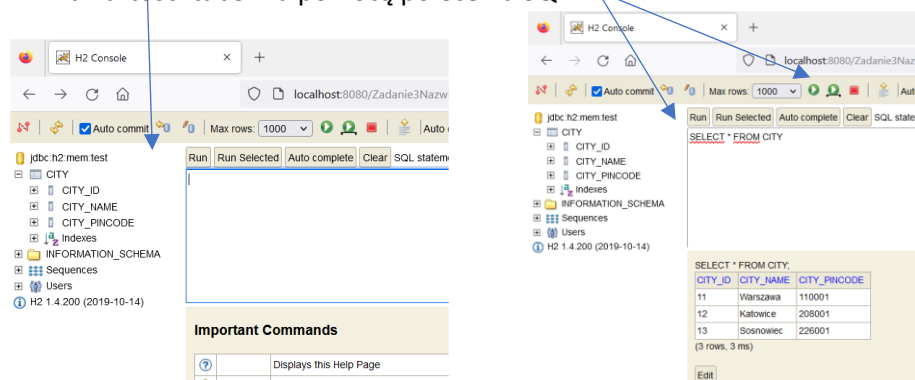
- g) Zbudowanie projektu – Clean and Build
 h) Uzupełnienie brakujących zależności gdy konieczne



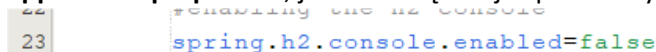
- i) Sprawdzenie działania panelu administracyjnego bazy danych – połączenie i podgląd bazy H2 – dane połączenia zgodne z plikiem application.properties – Test Connection



- j) Nawiązanie połączenia z bazą danych utworzoną przez SpringBooot H2 i sprawdzenie zawartości tabeli za pomocą polecenia SQL

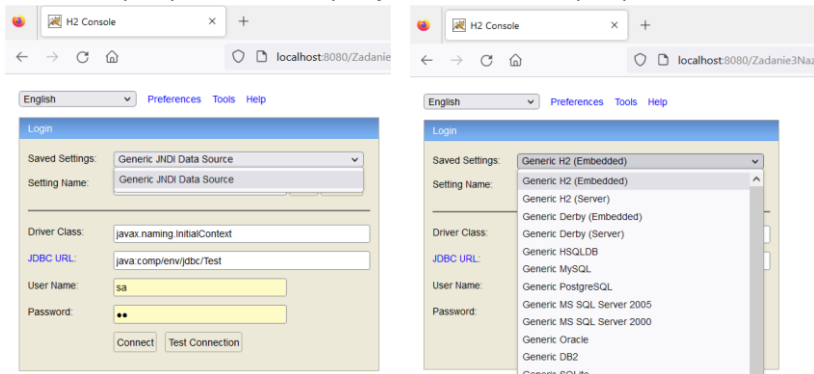


- k) Wyłączenie panelu administracyjnego po sprawdzeniu działania bazy – application.properties , jeżeli nie będzie już potrzebny



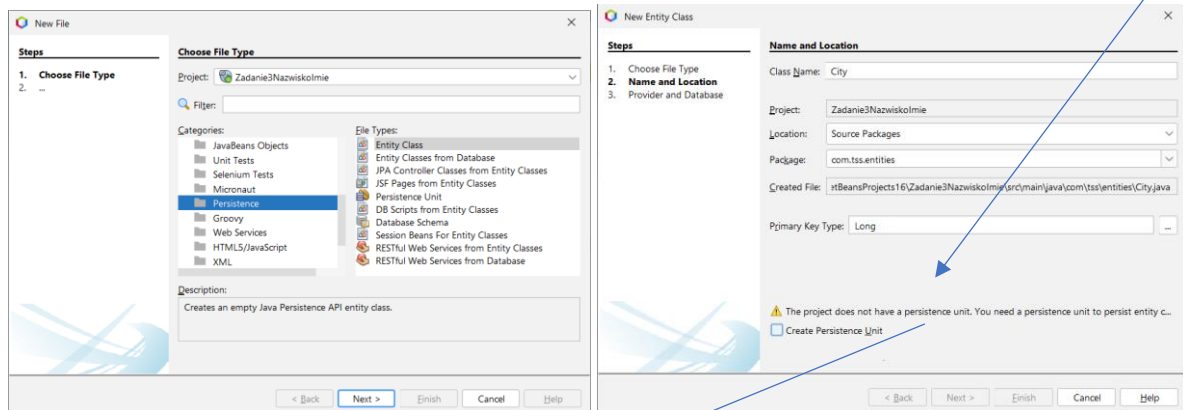
- a. **Uwaga:** W przypadku usunięcia skonfigurowanych, domyślnych połączeń do baz w konsoli H2 możliwe jest przywrócenie poprzez usunięcie pliku **.h2.server.properties** poprzez skasowanie pliku {userHome}/.h2.server.properties w folderze użytkownika

l) Lista domyślnych ustawień połączeń do baz danych panelu H2



5. Dodanie klasy encji i repozytorium

- a) Dodanie klasy encji **City** (Spring w aktualnych wersjach nie wymaga pliku persistence.xml)



Wygenerowany szablon klasy encji City.java

```
package com.tss.entities;

import java.io.Serializable;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;

@Entity
public class City implements Serializable {

    private static final long serialVersionUID = 1L;
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;
}
```

W przypadku innych środowisk lub braku kreatora encji należy przygotować klasę new Java Class i uzupełnić wymagany kod.

b) Uzupełnienie kodu encji City

```
@Entity
public class City {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    private Integer pincode;

    private Long population;

    public City() {}

    public City(String name, Integer pincode) {
        this.name = name;
        this.pincode = pincode;
    }

    public City(String name, Integer pincode, Long population) {
        this.name = name;
        this.pincode = pincode;
        this.population = population;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public Integer getPincode() {
        return pincode;
    }

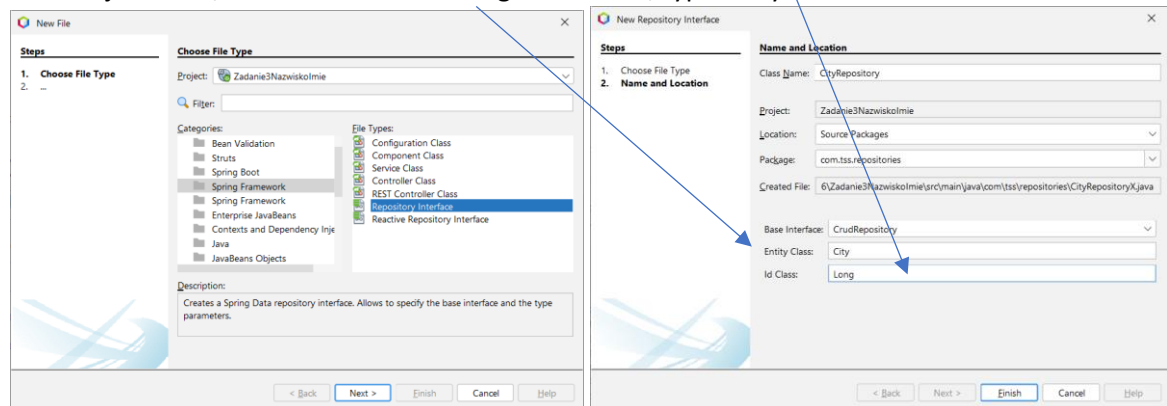
    public void setPincode(Integer pincode) {
        this.pincode = pincode;
    }

    public Long getPopulation() {
        return population;
    }

    public void setPopulation(Long population) {
        this.population = population;
    }

    @Override
    public String toString() {
        return "City{" + "id=" + id + ", name=" + name + ", pincode=" + pincode + ", population=" + population + '}';
    }
}
```

- c) Dodanie interfejsu repozytorium **CityRepository** za pomocą kreatora (z NB Spring Boot) lub bez jako new/Java Interface... - obsługiwana klasa, typ identyfikatora



- d) Uzupełnienie i modyfikacja kodu klasy **CityRepository**

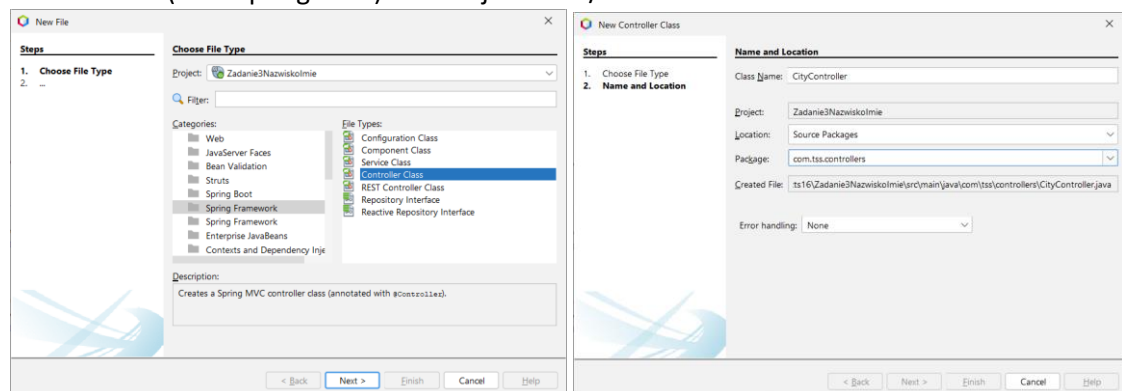
```
package com.tss.repositories;

import org.springframework.data.repository.CrudRepository;
import com.tss.entities.City;

public interface CityRepository extends CrudRepository<City, Long> {

}
```

- e) Dodanie klasy kontrolera **CityController** współpracującego z widokiem aplikacji, za pomocą kreatora (z NB Spring Boot) lub bez jako new/Java Java Class...



- f) Wygenerowany szablon kodu kontrolera **CityController**:

```
package com.tss.controllers;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.RequestMapping;

@Controller
public class CityController {

    @RequestMapping("/url")
    public String page(Model model) {
        model.addAttribute("attribute", "value");
        return "view.name";
    }

}
```

- g) Uzupełnienie kodu kontrolera **CityController**: **dodanie wymaganych poleceń import**, **wstrzyknięcie klasy repozytorium**, **wczytanie danych z repozytorium**, **zapisanie danych do modelu**, **wywołanie widoku**

```
package com.tss.controllers;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;

import com.tss.repositories.CityRepository;

import org.springframework.web.bind.annotation.RequestMapping;

@Controller
public class CityController {

    private final CityRepository cityRepository;

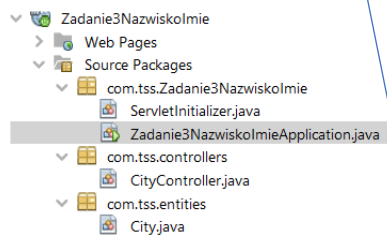
    @Autowired
    public CityController(CityRepository cityRepository) {
        this.cityRepository = cityRepository;
    }

    @RequestMapping("/")
    public String showUserList(Model model) {

        model.addAttribute("cities", cityRepository.findAll());

        return "/views/index.jsp";
    }
}
```

- h) Dodanie ścieżek pakietów do skanowania w klasie startowej **Zadanie3NazwiskoImieApplication** w celu odnalezienia ich przez Springa i adnotacji konfiguracyjnych



```
package com.tss.Zadanie3NazwiskoImie;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.autoconfigure.EnableAutoConfiguration;
import org.springframework.boot.autoconfigure.domain.EntityScan;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.data.jpa.repository.config.EnableJpaRepositories;
import org.springframework.transaction.annotation.EnableTransactionManagement;

@SpringBootApplication
@EnableAutoConfiguration
@ComponentScan(basePackages={"com.tss"})
@EnableJpaRepositories(basePackages="com.tss.repositories")
@EnableTransactionManagement
@EntityScan(basePackages="com.tss.entities")
public class Zadanie3NazwiskoImieApplication {

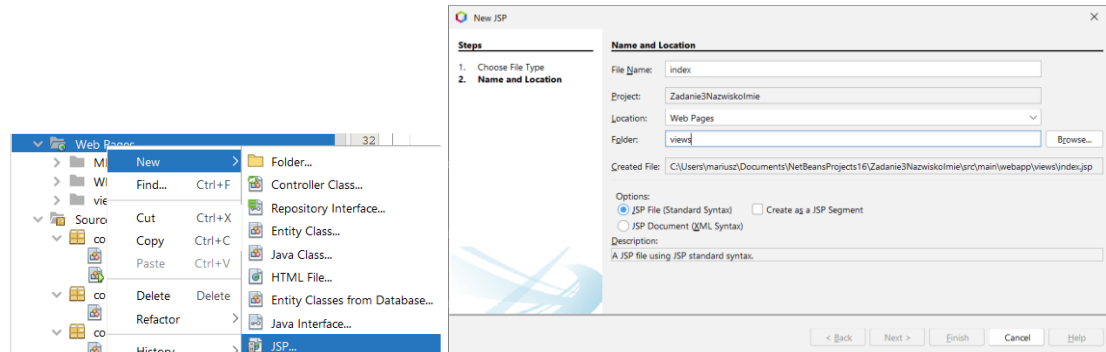
    public static void main(String[] args) {
        SpringApplication.run(primarySource: Zadanie3NazwiskoImieApplication.class, args);
    }
}
```

6. Prezentacja danych w postaci tabeli w widoku w technologii JSP - index.jsp

a) Dodanie zależności JSTL (do łatwiejszego tworzenia tabel w widokach JSP) – pom.xml

```
<scope>test</scope>
</dependency>
<dependency>
  <groupId>javax.servlet</groupId>
  <artifactId>jstl</artifactId>
</dependency>
</dependencies>
```

b) Dodanie widoku index.jsp w podfolderze views projektu



c) Dodanie informacji o autorze zadania, dodania obsługi JSTL, dodanie kodu generującego tabelę i odczytującego dane z modelu

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8"%>
<title>JSP Page</title>
</head>
<body>
<h1>Zadanie 3</h1>
<h2>Nazwisko Imię</h2>
<h2>Dane z wbudowanego zapytania @Repository</h2>
<table>
<thead>
<tr>
<th>ID</th>
<th>Nazwa</th>
<th>Kod</th>
<th>Populacja</th>
</tr>
</thead>
<tbody>
<c:forEach items="${cities}" var="city">
<tr>
<td>${city.id}</td>
<td>${city.name}</td>
<td>${city.pincode}</td>
<td>${city.population}</td>
</tr>
</c:forEach>
</tbody>
</table>
</body>
</html>
```

d) Efekt działania aplikacji po uruchomieniu

localhost:8080/Zadanie3Nazwiskolmie/

Zadanie 3

Nazwisko Imię

Dane z wbudowanego zapytania @Repository

ID	Nazwa	Kod	Populacja
1	Warszawa	40121	1110001
2	Katowice	40122	350001
3	Sosnowiec	40123	205001
4	Kraków	40124	32112
5	Zakopane	40125	26001
6	Szczecin	40128	120230
7	Gdańsk	40129	150345
8	Poznań	40131	26001

7. Dodanie zapytań raportujących do klasy repozytorium CityRepository – złożone zapytania niestandardowe rozszerzające możliwości repozytoriów

a) Zapytanie bazujące na nazwie metody

```
@Repository
public interface CityRepository extends CrudRepository<City, Long> {

    List<City> findAllByOrderByNameAsc();
}
```

b) Uzupełnienie kodu kontrolera

```
@RequestMapping("/")
public String showUserList(Model model) {

    model.addAttribute( attributeName: "cities", attributeValue: cityRepository.findAll());

    //Zapytanie budowane na podstawie składni nazwy metody
    model.addAttribute( attributeName: "citiesOrderAsc", attributeValue: cityRepository.findAllByOrderByNameAsc());
}
```

c) Uzupełnienie kodu widoku

```
</table>
<h2>Dane z zapytania na podstawie nazwy metody findAllByOrderByNameAsc</h2>
<table>
  <thead>
    <tr>
      <th>ID</th>
      <th>Nazwa</th>
      <th>Kod</th>
      <th>Populacja</th>
    </tr>
  </thead>
  <tbody>
    <:forEach items="${citiesOrderAsc}" var="city">
      <tr>
        <td>${city.id}</td>
        <td>${city.name}</td>
        <td>${city.pincode}</td>
        <td>${city.population}</td>
      </tr>
    </forEach>
  </tbody>
</table>
```

d) Zapytanie raportujące JPQL bazujące na klasie z konstruktorem **CityReport** – jeżeli standardowe mechanizmy repozytoriów nie wystarczają

e) Utworzenie klasy pomocniczej **CityReport** w pakiecie **com.tss.repositories** – kod klasy

```
public class CityReport {

    private String citySize="";
    private long cityCount=0;

    public CityReport(String citySize, long cityCount) {
        this.citySize=citySize;
        this.cityCount=cityCount;
    }

    public String getCitySize() {
        return citySize;
    }

    public void setCitySize(String citySize) {
        this.citySize = citySize;
    }

    public long getCityCount() {
        return cityCount;
    }

    public void setCityCount(long cityCount) {
        this.cityCount = cityCount;
    }

}
```

f) Uzupełnienie kodu klasy repozytorium **CityRepository**

```
@Repository
public interface CityRepository extends CrudRepository<City, Long> {

    List<City> findAllByOrderByNameAsc();

    //Wynik zapytania i klasa z konstruktorem CityReport - ważna wielkość znaków
    @Query("SELECT new com.tss.repositories.CityReport('Male', COUNT(c.id)) "
        + "FROM City AS c WHERE c.population<100000 ")
    List<CityReport> countCitiesPopulation();
}
```

g) Uzupełnienie kodu klasy kontrolera **CityController**

```
@RequestMapping("/")
public String showUserList(Model model) {

    model.addAttribute( attributeName: "cities", attributeValue: cityRepository.findAll());

    //Zapytanie budowane na podstawie składni nazwy metody
    model.addAttribute( attributeName: "citiesOrderByAsc", attributeValue: cityRepository.findAllByOrderByNameAsc());

    //Zapytanie raportujące JPQL
    List<CityReport> countCitiesPopulation= cityRepository.countCitiesPopulation();
    model.addAttribute( attributeName: "cityReport", attributeValue: countCitiesPopulation);
}
```

h) Uzupełnienie kodu widoku **index.jsp**

```
</table>
<h2>Dane z zapytania JPQL</h2>
<table>
    <thead>
        <tr>
            <th>Wielkość</th>
            <th>Liczba miast</th>
        </tr>
    </thead>
    <tbody>
        <c:forEach items="${cityReport}" var="cityReport">
            <tr>
                <td>${cityReport.citySize}</td>
                <td>${cityReport.cityCount}</td>
            </tr>
        </c:forEach>
    </tbody>
</table>
```

i) Zapytanie raportujące bazujące na projekcji na podstawie interfejsu **ICityReport** z użyciem pełnego natywnego SQL dla bazy danych H2

j) Utworzenie interfejsu pomocniczego **ICityReport** (new Java Interface) w pakiecie **com.tss.repositories** – kod interfejsu

```
package com.tss.repositories;

public interface ICityReport {

    String getCitySize();
    Long getCityCount();
}
```

k) Uzupełnienie kodu klasy repozytorium

```
List<CityReport> countCitiesPopulation();

//Wynik zapytania z użyciem Native Query - ważna wielkość znaków - można używać UNION
//i pełnej składni SQL - dane w kolumnach muszą być zamapowane przez AS zgodne z get w
//ICityRepository
@Query(value="SELECT 'Małe' AS citySize, COUNT(c.id) AS cityCount "
+ "FROM City AS c WHERE c.population<100000 "
+ "UNION ALL "
+ "SELECT 'Średnie', COUNT(c.id)"
+ "FROM City AS c WHERE c.population>=100000 AND c.population<200000 "
+ "UNION ALL "
+ "SELECT 'Duże', COUNT(c.id)"
+ "FROM City AS c WHERE c.population>=200000",nativeQuery = true)
List<ICityReport> countCitiesPopulationNativeQuery();
```

l) Uzupełnienie kodu klasy kontrolera

```
model.addAttribute( attributeName: "cityReport", attributeValue: countCitiesPopulation());

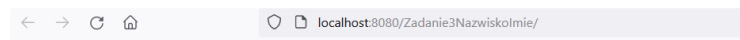
//Zapytanie raportujące native SQL
List<ICityReport> countCitiesPopulation2= cityRepository.countCitiesPopulationNativeQuery();
model.addAttribute( attributeName: "cityReport2", attributeValue: countCitiesPopulation2);

return "/views/index.jsp";
}
```

m) Uzupełnienie kodu widoku

```
</thead>
<tbody>
    <c:forEach items="${cityReport2}" var="cityReport2">
        <tr>
            <td>${cityReport2.citySize}</td>
            <td>${cityReport2.cityCount}</td>
        </tr>
    </c:forEach>
</tbody>
</table>
</body>
</html>
```

n) Efekt działania – wyniki powyższych zapytań w postaci tabel



Zadanie 3

Nazwisko Imię

Dane z wbudowanego zapytania @Repository

ID	Nazwa	Kod	Populacja
1	Warszawa	40121	1110001
2	Katowice	40122	350001
3	Sosnowiec	40123	205001
4	Kraków	40124	32112
5	Zakopane	40125	26001
6	Szczecin	40128	120230
7	Gdańsk	40129	150345
8	Poznań	40131	26001

Dane z zapytania na podstawie nazwy metody findAllByOrderByNameAsc

ID	Nazwa	Kod	Populacja
7	Gdańsk	40129	150345
2	Katowice	40122	350001
4	Kraków	40124	32112
8	Poznań	40131	26001
3	Sosnowiec	40123	205001
6	Szczecin	40128	120230
1	Warszawa	40121	1110001
5	Zakopane	40125	26001

Dane z zapytania JPQL

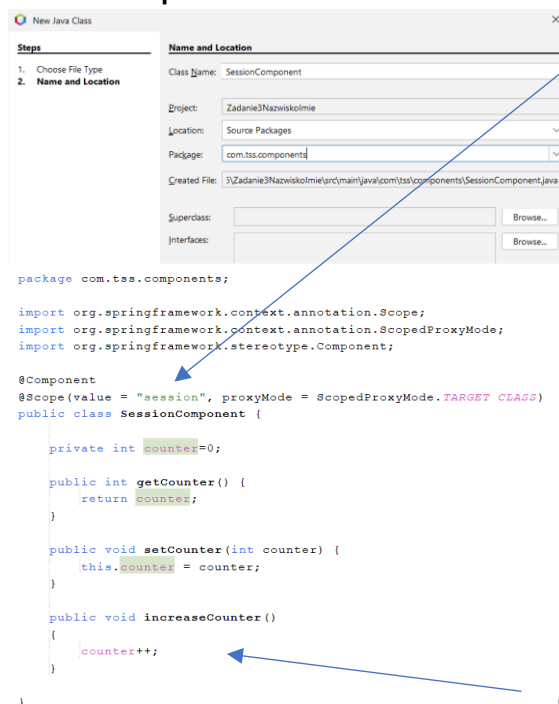
Wielkość Liczba miast
Male 3

Dane z zapytania Native Query

Wielkość Liczba miast
Male 3
Średnie 2
Duże 3

8. Dodanie licznika zapytań z wykorzystaniem komponentu o zasięgu sesji

- a) Utworzenie klasy komponentu **SessionComponent** o zasięgu sesji w pakiecie **com.tss.components**



Obsługa licznika zapytań

b) Wstrzyknięcie komponentu do klasy kontrolera **CityController** i dodanie poleceń **import**.

```
import com.tss.components.SessionComponent;

@Controller
public class CityController {

    private final CityRepository cityRepository;

    @Autowired
    SessionComponent sessionComponent;

    @Autowired
    public CityController(CityRepository cityRepository) {
        this.cityRepository = cityRepository;
    }

    @RequestMapping("/")
    public String showUserList(Model model) {

        model.addAttribute( attributeName: "cities", attributeValue: cityRepository.findAll());

        //Zapytanie budowane na podstawie składni nazwy metody
        model.addAttribute( attributeName: "citiesOrderAsc", attributeValue: cityRepository.findAllByNameAsc());
        sessionComponent.increaseCounter();
        //Zapytanie raportujące JPQL
        List<CityReport> countCitiesPopulation= cityRepository.countCitiesPopulation();
        model.addAttribute( attributeName: "cityReport1", attributeValue: countCitiesPopulation);
        sessionComponent.increaseCounter();
        //Zapytanie raportujące native SQL
        List<ICityReport> countCitiesPopulation2= cityRepository.countCitiesPopulationNativeQuery();
        model.addAttribute( attributeName: "cityReport2", attributeValue: countCitiesPopulation2);
        sessionComponent.increaseCounter();

        model.addAttribute( attributeName: "counterValue", attributeValue: sessionComponent.getCounter());
        return "/views/index.jsp";
    }
}
```

c) Zwiększanie licznika zapytań w klasie kontrolera w po każdym wywołaniu metody z repozytorium i dodanie informacji o stanie licznika do modelu.

d) Dodanie informacji o liczbie zapytań do bazy w widoku **index.jsp**

```
<td>${cityReport2.citySize}</td>
<td>${cityReport2.cityCount}</td>
</tr>
</c:forEach>
</tbody>
</table>
<h2>Licznik zapytań: ${counterValue}</h2>
</body>
</html>
```

e) Efekt działania – komponent sesyjnego - przechowuje liczbę zapytań do bazy wykonaną przez aplikację. Stan licznika jest przechowywany w obrębie sesji. Fragment widoku ze stanem licznika – kolejne odświeżenia strony.

```
1 Warszawa 40121 1110001
5 Zakopane 40125 26001
```

Dane z zapytania JPQL

Wielkość Liczba miast
Małe 3

Dane z zapytania Native Query

Wielkość Liczba miast
Małe 3
Średnie 2
Duże 3

Licznik zapytań: 3

```
1 Warszawa 40121 1110001
5 Zakopane 40125 26001
```

Dane z zapytania JPQL

Wielkość Liczba miast
Małe 3

Dane z zapytania Native Query

Wielkość Liczba miast
Małe 3
Średnie 2
Duże 3

Licznik zapytań: 24

```
1 Warszawa 40121 1110001
5 Zakopane 40125 26001
```

Dane z zapytania JPQL

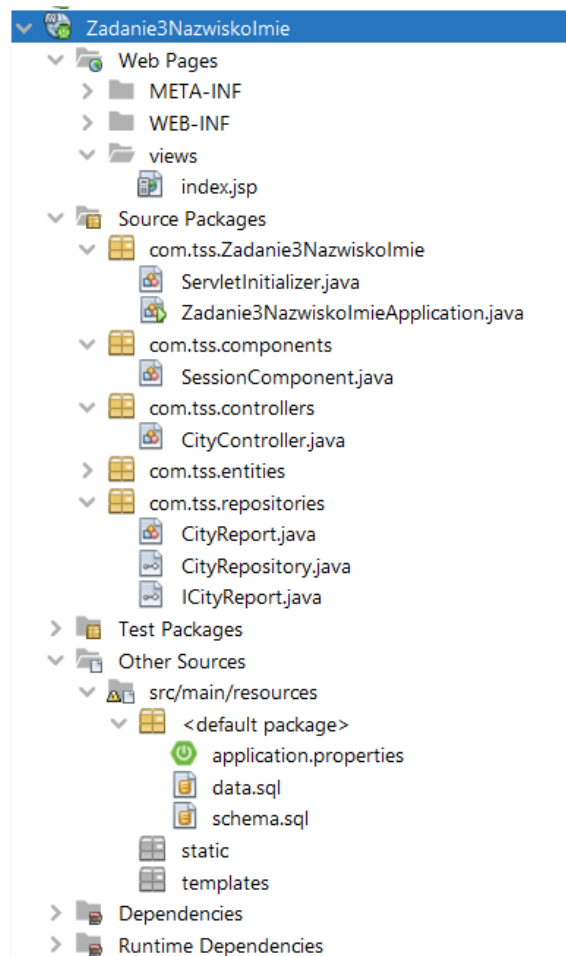
Wielkość Liczba miast
Małe 3

Dane z zapytania Native Query

Wielkość Liczba miast
Małe 3
Średnie 2
Duże 3

Licznik zapytań: 6

9. Struktura opracowanego projektu:



10. Dowolne własne modyfikacje niewpływające na główną część zadania.

11. Przesłanie wyników na platformę e-learning:

a) Sprawdzenie działania projektu z poziomu środowiska IDE i pliku .war i bezpośrednio na serwerze poprzez wgranie pliku .war do folderu **D:\apache-9-0.46\webapps**

b) **Minimalizacja wielkości archiwum przez usunięcie plików roboczych generowanego projektu**

nty > NetBeansProjects16 > Zadanie3NazwiskoImie > target >					Przeszukaj: target	
Nazwa		Data modyfikacji	Typ	Rozmiar		
classes		2023.02.08 16:19	Folder plików			
generated-sources		2023.02.08 16:19	Folder plików			
maven-archiver		2023.02.08 16:20	Folder plików			
surefire-reports		2023.02.08 16:20	Folder plików			
test-classes		2023.02.08 16:19	Folder plików			
Zadanie3NazwiskoImie-0.0.1-SNAPSHOT		2023.02.08 16:20	Folder plików			
Zadanie3NazwiskoImie-0.0.1-SNAPSHOT.war		2023.02.08 16:20	Plik WAR	38 557 KB		
Zadanie3NazwiskoImie-0.0.1-SNAPSHOT.war.original		2023.02.08 16:20	Plik ORIGINAL	30 877 KB		

c) Przesłanie własnego projektu (kompletny folder projektu z całą zawartością i plikiem .war) w postaci archiwum **Zadanie3NazwiskoImie.zip,rar,7z**.

d) Dołączyć informację o środowisku: wersja serwera Tomcat i JavaSDK (w pliku opis.txt) jeżeli inne niż w zadaniu.